UAM

Programación de Aplicaciones de Escritorio

Evaluación 3 – Corte 2

Informe de la Tarea sumativa 2

Procesamiento de pedidos con cancelación

Dato	Valor
Modalidad	Trabajo grupal (3 integrantes)
Docente	**[Pendiente]**
Fecha de las pruebas	29 de octubre de 2026
Fecha de entrega	**[Pendiente]**
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2

1. Identificación del caso e integrantes

Caso: Tarea sumativa 2 – Procesamiento de pedidos con cancelación (sesión 2, indicadores IL3.1 e IL3.3). **Módulo:** Procesamiento_de_pedidos. **Responsable principal del caso:** Erving Miranda.

Tabla 1. Integrantes del grupo

Integrante	Usuario o autor en Git
Erving Miranda	Erving Miranda
[Pendiente: nombre completo]	**[Pendiente]**
[Pendiente: nombre completo]	**[Pendiente]**

En el historial del repositorio aparecen cuatro autores: Erving Miranda, Andre / André Guido, ValcocerLopez (Victor Mateo Alcocer López) y Argonauta0. La guía indica grupos de tres integrantes, por lo que la lista oficial debe confirmarse antes de la entrega.

2. Objetivo y descripción del funcionamiento

El objetivo es procesar 30 pedidos, uno por uno, en segundo plano. Cada pedido simula un trabajo de 200 ms y solo se cuenta como completado cuando termina esa espera. El usuario puede cancelar el procesamiento en cualquier momento sin detener el programa a la fuerza.

La ventana tiene los botones **Iniciar** y **Cancelar**, una barra de progreso, el contador «Pedidos completados» y un registro de procesamiento. La aplicación maneja seis estados propios, distintos de Thread.State:

Tabla 2. Estados de la aplicación

Estado	Significado
Listo	La ventana se abrió y todavía no se ha iniciado un procesamiento.
En ejecución	El hilo trabajador está procesando pedidos.
Cancelando	Se pidió cancelar y se espera que el trabajador termine.
Cancelado	El trabajador terminó después de cancelar, con menos de 30 pedidos.
Completado	Se procesaron los 30 pedidos.
Error	El trabajador terminó antes de tiempo sin que se pidiera cancelar.

3. Hilos y cancelación cooperativa

3.1 Hilo trabajador

La clase ProcesadorPedidos implementa Runnable y contiene el ciclo de los 30 pedidos. Al pulsar Iniciar, el controlador crea un Thread nuevo con esa tarea y llama a start(). Se crea un hilo nuevo en cada ejecución porque un Thread que ya terminó no puede volver a iniciarse.

3.2 Cancelación con interrupt()

Cancelar llama a interrupt() sobre el hilo trabajador; nunca se usa Thread.stop(). El trabajador responde a la solicitud en dos puntos: antes de comenzar cada pedido revisa isInterrupted(), y si la interrupción llega mientras está en Thread.sleep(200), esa espera lanza InterruptedException. En ambos casos el ciclo termina y el pedido que estaba en curso no se cuenta.

```
for (int numeroPedido = 1; numeroPedido <= TOTAL_PEDIDOS; numeroPedido++) {
    // Revisa entre un pedido y otro si se pidió cancelar con interrupt().
    if (Thread.currentThread().isInterrupted()) return;

    Thread.sleep(DURACION_PEDIDO_MS);   // lanza InterruptedException si llega interrupt()

    // El pedido solo se cuenta después de terminar su espera completa.
    controlador.registrarPedidoCompletado(numeroPedido);
}
```

3.3 Espera asíncrona de la terminación

Después de pedir la cancelación, la aplicación no habilita otro inicio hasta confirmar que el trabajador terminó. Para eso un segundo hilo, el vigilante, llama a join() sobre el trabajador. join() deja esperando al hilo que lo llama; por eso no se llama desde JavaFX, ya que la ventana se congelaría. Cuando join()

regresa, el vigilante usa Platform.runLater() para mostrar el resultado final (Completado, Cancelado o Error) y volver a habilitar Iniciar.

3.4 Actualizaciones seguras de la interfaz

Cada vez que termina un pedido, el trabajador llama a registrarPedidoCompletado(), que usa Platform.runLater() para aumentar el contador, mover la barra de progreso y escribir en el registro desde el hilo de JavaFX. Como el contador solo se modifica en ese hilo, no se necesitan variables atómicas ni volatile. Platform.runLater() ejecuta las tareas en el orden en que se encolan, por lo que el resultado final siempre se muestra después del último pedido registrado.

3.5 Inicio duplicado y cierre de la ventana

Mientras exista un trabajador, Iniciar está deshabilitado y el método iniciarProcesamiento() también lo comprueba. Si se cierra la ventana con un procesamiento activo, el evento de cierre se consume, se solicita la cancelación y la ventana se cierra cuando el vigilante confirma que el trabajador terminó. El hilo de JavaFX nunca queda bloqueado esperando.

4. Resultados esperados y obtenidos

Tabla 3. Pruebas de aceptación del caso 2

Escenario	Resultado esperado	Resultado obtenido	Estado
Estado inicial	Listo, 0 / 30 y Cancelar deshabilitado	Listo, 0 / 30, barra vacía y Cancelar deshabilitado (Figura 1)	Cumple
Ejecución completa	30 / 30, progreso 100 % y Completado	30 / 30, barra llena, estado Completado y 30 avisos en el registro (Figura 3)	Cumple
Interfaz operativa	Procesar fuera del hilo de JavaFX	La ventana respondió y se actualizó durante la ejecución; a los 3 s mostraba 14 / 30 (Figura 2)	Cumple
Cancelación durante una espera	Contador entre 0 y 30, estable al terminar; estado Cancelado	Se detuvo en 11 / 30 con estado Cancelado; 3 s después la ventana no había cambiado (Figura 4)	Cumple
Inicio duplicado	No crear otro trabajador	Iniciar quedó deshabilitado durante la ejecución; el clic adicional no tuvo efecto (Figura 2)	Cumple
Reinicio después de cancelar	Nueva tarea con el contador restablecido	Tras cancelar en 8 / 30, la nueva ejecución empezó desde 0 con el registro limpio (Figuras 5 y 6)	Cumple
Cierre con la tarea activa	Cancelar, cerrar sin bloquear y sin trabajadores activos	Se cerró la ventana en 9 / 30; se cerró al terminar el trabajador y el proceso Java finalizó con código 0 (Figura 7)	Cumple

Sobre el cierre: los hilos trabajador y vigilante no son daemon, de modo que si alguno siguiera vivo el proceso Java no podría terminar. Que el proceso finalizara con código 0 inmediatamente después del cierre confirma que no quedaron trabajadores activos.

Estados Cancelando y Error: el estado Cancelando dura menos de 200 ms, porque interrupt() despierta inmediatamente al trabajador, por lo que no fue posible capturarlo en pantalla. Tanto Cancelando como Error (que no puede provocarse desde los botones) se verificaron con las pruebas automáticas descritas a continuación.

Tabla 4. Pruebas automáticas: 13 ejecutadas y 13 aprobadas (29/10/2026)

Prueba JUnit	Qué comprueba
completaTreintaPedidosDeDoscientosMilisegundos	Los 30 pedidos se procesan en orden y cada uno espera al menos 190 ms.
respetaInterrupcionAntesDeComenzar	Si el hilo ya está interrumpido, no se procesa ningún pedido.
cancelaDuranteLaEspera	Al interrumpir durante sleep() el hilo termina y el pedido incompleto no se cuenta.
cancelaEntrePedidos	Una interrupción entre dos pedidos impide comenzar el siguiente.
comienzaListo / completaYPermiteReiniciar	Estado inicial y ciclo completo con reinicio desde cero (ventana real).
impideDuplicadosYReinicioMientrasCancela	Inicios repetidos no crean otro hilo; se muestra Cancelando y luego Cancelado.
cancelaConProgresoParcial	El resultado cancelado conserva exactamente los pedidos completados.
cancelarTrasElUltimoPedidoConservaCompletado	Una cancelación tardía, después del pedido 30, termina en Completado.
errorLiberaTrabajadorSinEsperarLaAlerta	Una interrupción ajena a Cancelar produce el estado Error y libera los botones.
cierraDeFormaSeguraDuranteProcesamiento / aplicacionInstalaCierreAntesDeMostrar / cierraSinIniciar	Cierre durante el procesamiento (pasa por Cancelando y espera al trabajador), cierre inmediato tras abrir y cierre sin iniciar.

5. Pruebas realizadas y capturas

Las capturas corresponden a ejecuciones reales de la aplicación realizadas el 29 de octubre de 2026.

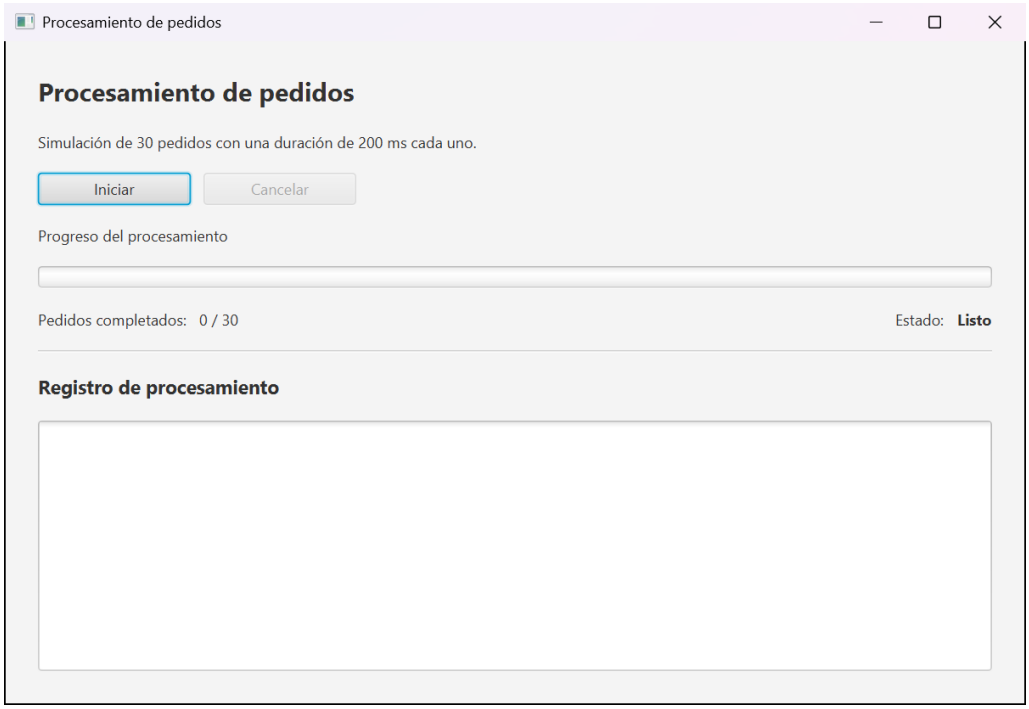


Figura 1. *Estado inicial: Listo, 0 / 30 y Cancelar deshabilitado.*

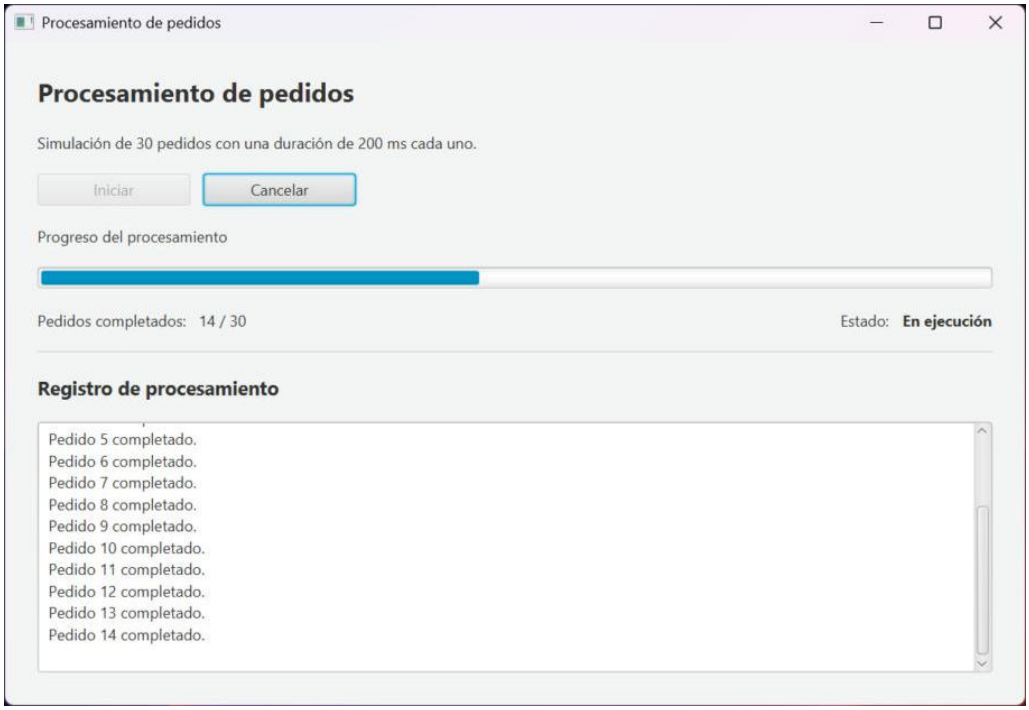


Figura 2. *En ejecución: 14 / 30 pedidos; Iniciar deshabilitado para impedir un inicio duplicado.*

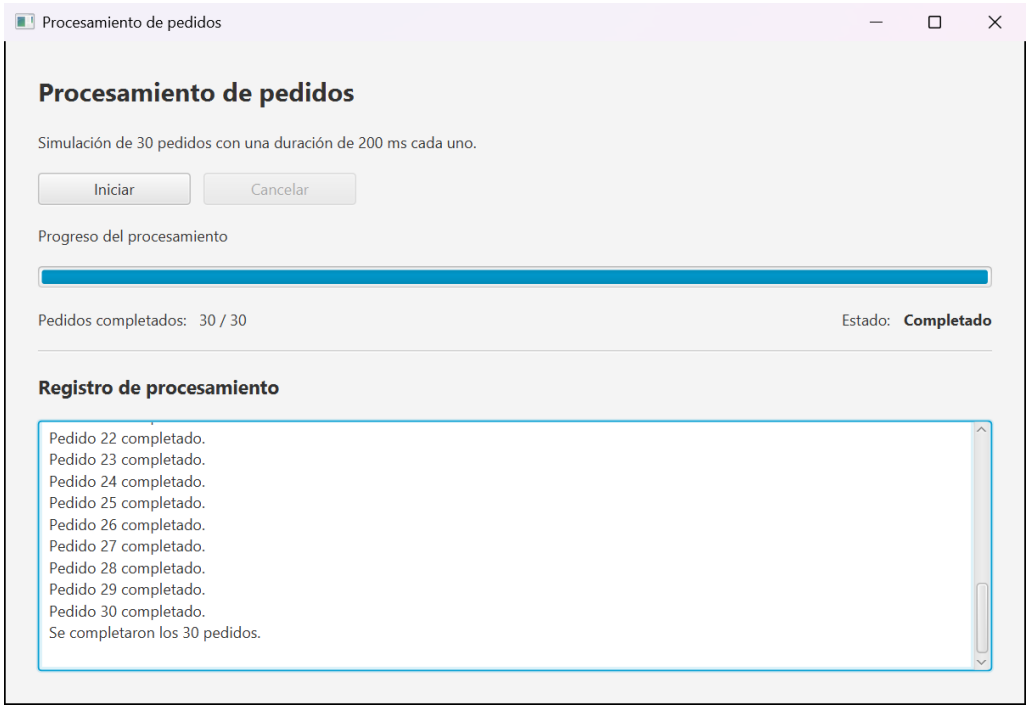


Figura 3. *Ejecución completa: 30 / 30, progreso del 100 % y estado Completado.*

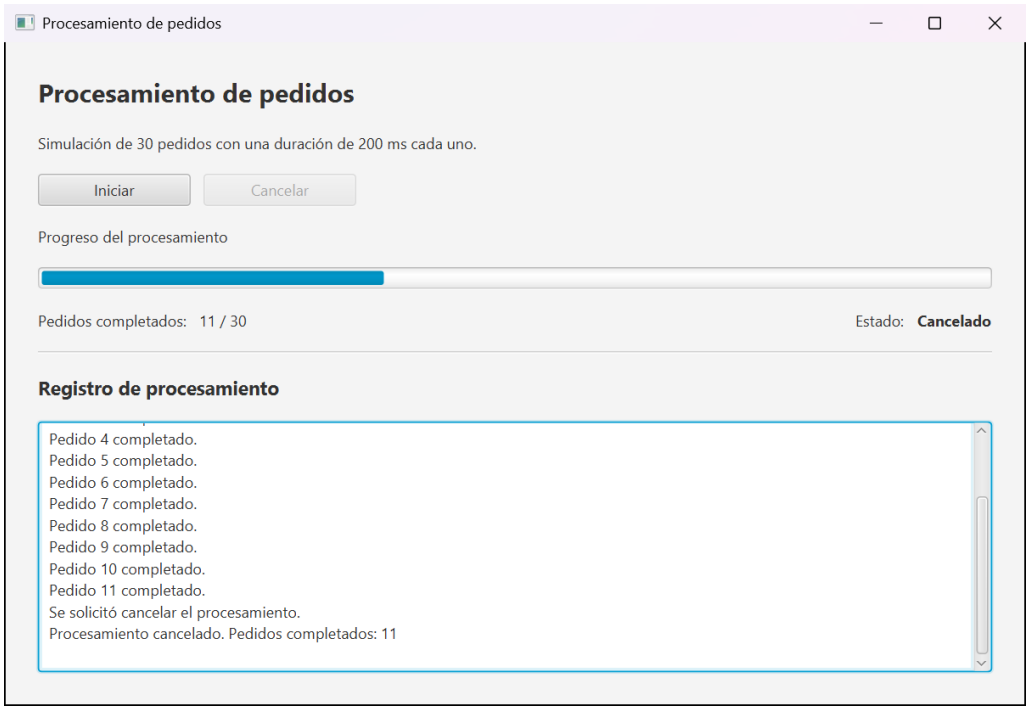


Figura 4. *Cancelación durante el procesamiento: 11 / 30 y estado Cancelado. El contador no cambió después.*

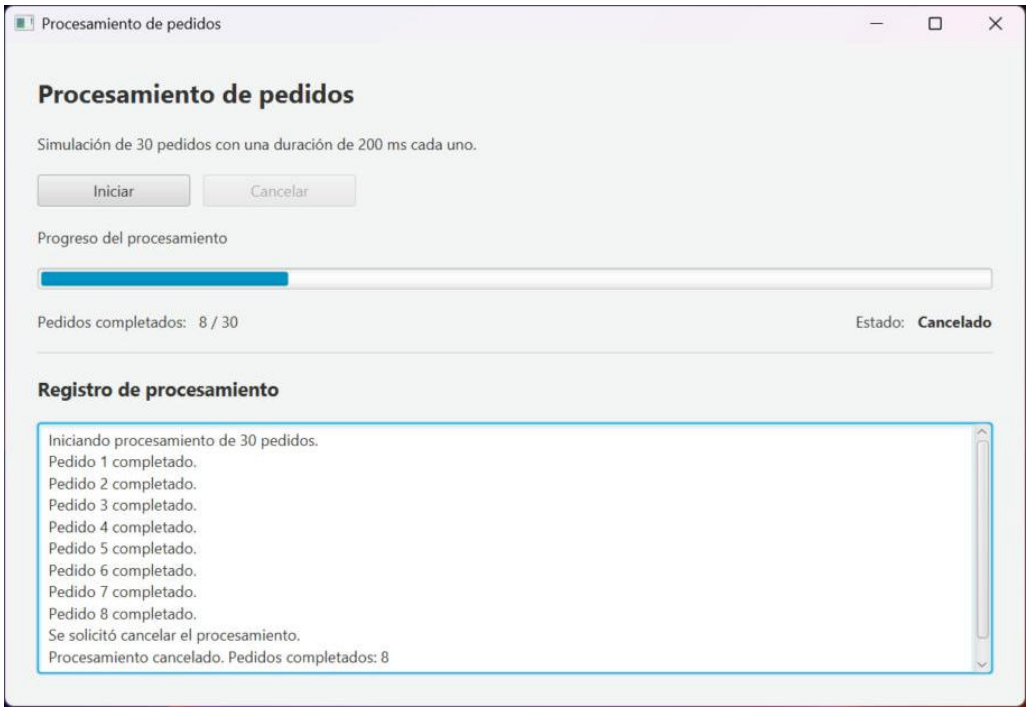


Figura 5. *Segunda cancelación (8 / 30) antes de volver a iniciar.*

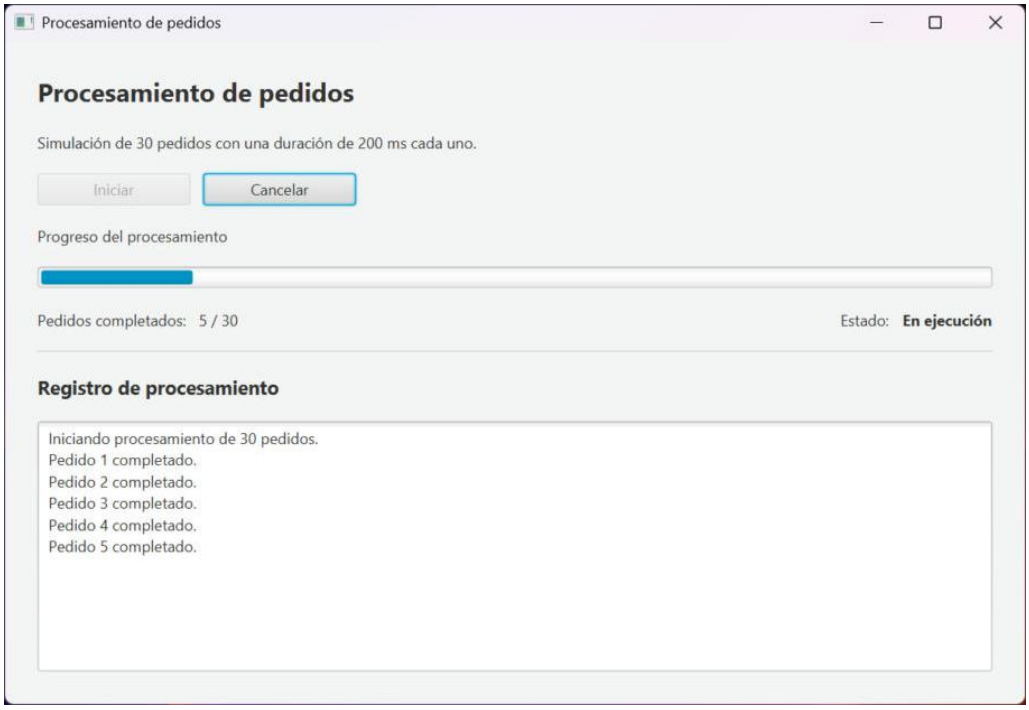


Figura 6. *Reinicio después de cancelar: nueva ejecución desde 0 y registro restablecido.*

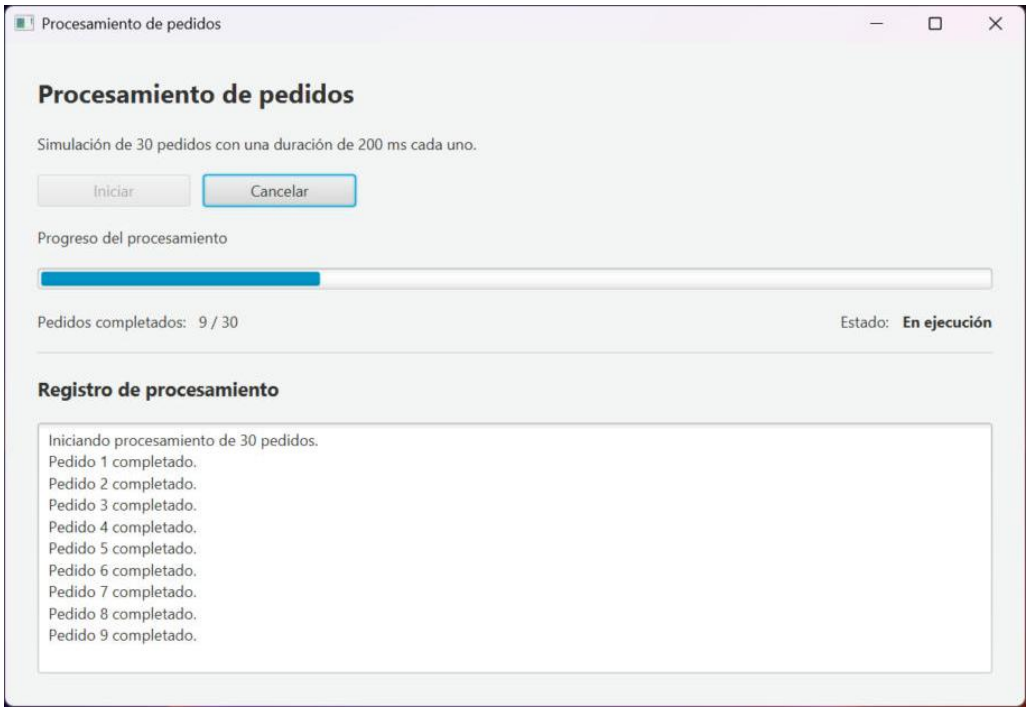


Figura 7. *Ventana en ejecución (9 / 30) justo antes de cerrarla; se cerró al terminar el trabajador y el proceso finalizó con código 0.*

6. Distribución de responsabilidades

La siguiente tabla resume los aportes que pueden comprobarse en el historial de Git del repositorio. La distribución oficial de responsabilidades (interfaz, lógica concurrente, pruebas y documentación) debe ser confirmada por el grupo.

Tabla 5. Aportes registrados en Git

Autor en Git	Aportes registrados en el repositorio
ValcocerLopez	Estructura inicial del monorepo, menú principal, corrección de los pom.xml y del controlador de la caja; fusión de la solicitud #1 (Reserva_Concurrente).
Andre / André Guido	Caso 1: vista reserva-view.fxml, hojas de estilo, clases Inventario, AgenteReserva y ReservaController; fusión de la solicitud #2 (caja_pedidos).
Argonauta0	Caso 3: vista caja-ventas-view.fxml, CajaVentasController, CajaVentasApplication, modelos CajaCompartida y ProcesadorVentasTask, Launcher y comentarios.
Erving Miranda	Caso 2: PedidosController, ProcesadorPedidos, PedidosApplication, vista pedidos-view.fxml y pruebas JUnit; revisión final del conjunto, README e informes.

Tabla 6. Responsabilidades declaradas por el grupo

Integrante	Responsabilidad en este caso
Erving Miranda	Lógica concurrente, interfaz, pruebas JUnit, pruebas manuales e informe del

	caso 2.
[Pendiente]	**[Pendiente]**
[Pendiente]	**[Pendiente]**

7. Conclusiones

La aplicación procesa los 30 pedidos fuera del hilo de JavaFX y mantiene la interfaz operativa. La cancelación es cooperativa: interrupt() solicita detenerse, el trabajador lo comprueba entre pedidos y durante la espera, y ningún pedido incompleto se cuenta. El hilo vigilante, con join(), confirma la terminación sin bloquear la ventana antes de permitir un reinicio o de cerrarla. Con Thread, Runnable, sleep(), interrupt() y join() fue suficiente para cumplir todos los requisitos sin mecanismos adicionales.

8. Repositorio y datos de ejecución

Tabla 7. Datos de ejecución

Dato	Valor
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2
Rama revisada	fix/revision-entrega (pendiente de integrar a master mediante revisión del grupo)
Módulo	Procesamiento_de_pedidos (clase de inicio ni.edu.uam.procesamiento_de_pedidos.Launcher)
JDK	Microsoft OpenJDK 21.0.13 (JDK 21)
JavaFX	21.0.6 (javafx-controls y javafx-fxml)
IDE	IntelliJ IDEA 2026.2.3
Sistema operativo	Windows 11 Pro
Configuración de IntelliJ	Caso 2 - Procesamiento de pedidos
Forma de ejecución en estas pruebas	Línea de comando equivalente a la configuración de IntelliJ (java -p <JavaFX 21.0.6 y target/classes> -m <módulo>/<Launcher>). El caso 1 se ejecutó directamente desde IntelliJ; para este caso se usó la misma línea porque el monitor donde estaba IntelliJ se desconectó durante la sesión de pruebas.

Para ejecutar desde IntelliJ IDEA: abrir la carpeta raíz del repositorio, seleccionar un SDK 21, elegir la configuración «Caso 2 - Procesamiento de pedidos» o «Menu principal» en la barra superior y pulsar Run. Las configuraciones se incluyen en la carpeta .run/ del repositorio.